Design and Implementation of a Basic 16-bit RISC Processor

on FPGA — Boolean Board (Xilinx Spartan-7)

Course	FPGA Based Design
Platform	Boolean Board — Xilinx XC7S50 Spartan-7
Tool	Xilinx Vivado 20xx.x
Language	Verilog HDL
Architecture	8-bit data, 16-bit fixed instruction, 8 registers, sequential execution
Difficulty	Advanced — Multi-module Processor Design

Table of Contents

SL No.	Specification	Page No.
1	Introduction	02
2	Objectives	02
3	Processor Specifications	03
4	Instruction Set Architecture (ISA)	03-04
5	Instruction Execution Flow	05
6	Module Design & Implementation	05-14
7	Module Synthesize Block Diagram	14-15
8	Block Diagram	16
9	State Diagram	17
10	Mandatory Test Program	18
11	Simulation Results	18-19
12	FPGA Synthesis & Implementation	19-22
13	Conclusion	23

1. Introduction

A Reduced Instruction Set Computer (RISC) processor is the foundational architecture behind virtually every modern processor. The defining principles of RISC design — fixed-width instructions, load/store memory model, a small uniform register file, and single-cycle or pipelined execution — make it an ideal subject for FPGA implementation. Building a processor from scratch requires integrating all major concepts of digital design: instruction set architecture, datapath design, control logic, register file organization, ALU construction, and memory interfacing.

This assignment requires students to design, simulate, and deploy a basic 16-bit RISC processor on the Boolean Board (Xilinx XC7S50 Spartan-7). The processor implements a custom ISA supporting arithmetic, comparison, and memory operations over an 8-bit datapath. All instructions are encoded as fixed 16-bit words. Execution is sequential — one instruction completes before the next is fetched. No branch or jump instructions are required.

This is the most complex assignment in the series, requiring students to integrate multiple independently designed modules — Program Counter, Instruction Memory, Register File, ALU, Multiplier, Data Memory, and Control Unit — into a single working processor top module, verify it with a mandatory test program, and demonstrate it on hardware.

2. Objectives

The primary objectives of this laboratory assignment are:

- Design a custom 16-bit fixed-width ISA supporting arithmetic, comparison, and load/store memory operations.
- Implement a multi-module RISC processor datapath in Verilog HDL: PC, instruction memory, register file, ALU, multiplier, data memory, and control unit.
- Design a synchronous control unit (decoder) that generates all datapath control signals from the 4-bit opcode.
- Execute the mandatory test program correctly and verify results against expected register and memory values.
- Synthesize, implement, and demonstrate the processor on the Boolean Board using switches, LEDs, and/or 7-segment display.
- Produce simulation waveforms showing correct fetch-decode-execute behavior for each instruction in the test program.
- Document the design with a block diagram, ISA encoding table, datapath description, and implementation report.

3. Processor Specifications

The processor conforms to the following fixed architectural parameters. These constraints must not be modified.

Parameter	Specification
Data Width	8-bit — all registers, ALU operands, and memory data are 8-bit wide
Instruction Width	16-bit fixed width — all instructions occupy exactly 16 bits
General-Purpose Registers	8 registers — R0 through R7, each 8-bit wide
Program Counter (PC)	Increments by 1 after each instruction; freezes on HALT; resets to 0 on rst
Instruction Memory	ROM / BRAM — preloaded with the program; addressed by PC (8-bit address)
Data Memory	RAM / BRAM — 512 locations, each 8-bit wide; 9-bit address space
Execution Model	Sequential, non-pipelined — one instruction completes before the next is fetched
Supported Operations	NOP, ADD, SUB, MUL, CMP, MOV, LOAD, STORE, HALT
Comparison Flags	Z (zero/equal), G (greater), L (less) — updated by CMP instruction only
Clock	Positive edge triggered; all state transitions on rising edge
Reset	Synchronous, active-HIGH — clears PC, IR, registers, flags

4. Instruction Set Architecture (ISA)

4.1 Instruction Formats

All instructions are exactly 16 bits wide. Two instruction formats are defined based on the operation type:

R-Type — Arithmetic, Logic, and Comparison Instructions

[15:12] opcode	[11:9] rd	[8:6] rs1	[5:3] rs2	[2:0] unused
4 bits	3 bits — destination register	3 bits — source register 1	3 bits — source register 2	3 bits — set to 000

Used by: ADD, SUB, MUL, CMP, MOV, NOP.

M-Type — Memory Instructions

[15:12] opcode	[11:9] rd / rs	[8:0] address
4 bits	3 bits — destination (LOAD) or source (STORE)	9 bits — data memory address (range 0 to 511)

Used by: LOAD, STORE

4.2 Opcode Encoding and Instruction Semantics

Opcode	Mnemonic	Operation	Description
0000	NOP	No operation	The processor advances PC by 1. No register or memory is modified. Used for pipeline alignment or intentional delay.
0001	ADD	Rd <= Rs1 + Rs2	8-bit unsigned addition. Result written to destination register Rd. Carries beyond 8 bits are discarded.
0010	SUB	Rd <= Rs1 - Rs2	8-bit unsigned subtraction. Result written to Rd. Borrows beyond 8 bits are discarded.
0011	MUL	Rd <= Rs1 x Rs2	8-bit multiplication. The lower 8 bits of the 16-bit product are written to Rd. A dedicated multiplier module is required.
0100	CMP	Set Z, G, L flags	Compares Rs1 and Rs2. Sets Z=1 if Rs1==Rs2, G=1 if Rs1>Rs2, L=1 if Rs1<Rs2. Rd field is unused. No register is written.
0101	MOV	Rd <= Rs1	Copies the value of register Rs1 into register Rd. Rs2 field is unused.
0110	LOAD	Rd <= MEM[address]	Reads the 8-bit value at the 9-bit data memory address and writes it into register Rd.
0111	STORE	MEM[address] <= Rs	Writes the 8-bit value in register Rs (encoded in bits [11:9]) to the 9-bit data memory address.
1111	HALT	Freeze PC; stop fetch	Stops processor execution. PC is frozen. The processor remains in HALT state until reset is asserted.

4.3 Comparison Flags

The CMP instruction updates three 1-bit flag registers. These flags are not directly accessible via instructions in this ISA but must be observable on FPGA LEDs for demonstration purposes.

Flag	Condition	Description
Z	Rs1 == Rs2	Zero flag — set HIGH when the two compared operands are equal.
G	Rs1 > Rs2	Greater flag — set HIGH when Rs1 is strictly greater than Rs2.
L	Rs1 < Rs2	Less flag — set HIGH when Rs1 is strictly less than Rs2.

Only one flag is HIGH at a time. All three flags are cleared to zero at reset and updated exclusively by the CMP instruction.

5. Instruction Execution Flow

Every instruction — regardless of type — must pass through the following six-stage sequential execution flow. Each stage occupies one clock cycle in the non-pipelined implementation:

Stage	Name	Action
1	Fetch	IR <= InstrMem[PC]. The 16-bit instruction at the current PC address is loaded into the Instruction Register.
2	Decode	The Control Unit decodes bits [15:12] (opcode) and generates RegWrite, MemRead, MemWrite, ALUSrc, ALUOp, and WBSel control signals.
3	Read Operands	The Register File outputs the values of Rs1 (bits [8:6]) and Rs2 (bits [5:3]). For M-type instructions, the 9-bit address (bits [8:0]) is extracted.
4	Execute	The ALU or Multiplier performs the required operation. For CMP, flags Z, G, L are updated. For MOV, the Rs1 value is passed through unchanged.
5	Memory Access	For LOAD: DataMem[address] is read. For STORE: Rs value is written to DataMem[address]. All other instructions skip this stage.
6	Write Back	The result (from ALU, Multiplier, or Data Memory) is written into the destination register Rd. CMP and STORE do not write to any register.

After Write Back, PC is incremented by 1 (PC <= PC + 1) and the next instruction is fetched. The HALT instruction freezes PC and prevents further fetching.

6. Module Design & Implementation

6.1 Program Counter (PC)

The Program Counter is an 8-bit synchronous register. On each rising clock edge, if rst is asserted, it resets to 8'h00. If pc_write is asserted by the control unit, it increments by 1. During HALT, pc_write is de-asserted, freezing the PC.

Module :- pc.v

```
module pc (
    input clk, pc_write, rst,
    output reg [7:0] pc_out
);
    always @(posedge clk) begin
        if (rst)
            pc_out <= 8'h00;
        else if (pc_write)
            pc_out <= pc_out + 8'h01;
    end
endmodule
```

Figure 6.1 :- Program Counter Module

6.2 Instruction Memory

The Instruction Memory is implemented as a 256x16-bit synchronous ROM. On each rising clock edge, when imem_read is asserted, the 16-bit instruction at address addr[7:0] (the current PC value) is latched into the output register instr[15:0]. The memory is preloaded using initial blocks with hardcoded hex values corresponding to the mandatory test program.

The hex encodings for the test program are: mem[0]=16'h620A (LOAD R1,10), mem[1]=16'h640B (LOAD R2,11), mem[2]=16'h1650 (ADD R3,R1,R2), mem[3]=16'h2850 (SUB R4,R1,R2), mem[4]=16'h3AD0 (MUL R5,R3,R2), mem[5]=16'h7A14 (STORE R5,20), mem[6]=16'h40E0 (CMP R3,R4), mem[7]=16'hF000 (HALT).

Module :- instr_mem.v

```
module instr_mem (
    input clk, imem_read,
    input [7:0] addr,
    output reg [15:0] instr
);
    reg [15:0] mem [0:255];
    integer i;
    initial begin
        for (i = 0; i < 256; i = i + 1)
            mem[i] = 16'h0000;

        mem[0] = 16'h620A;
        mem[1] = 16'h640B;
        mem[2] = 16'h1650;
        mem[3] = 16'h2850;
        mem[4] = 16'h3AD0;
        mem[5] = 16'h7A14;
        mem[6] = 16'h40E0;
        mem[7] = 16'hF000;
    end

    always @(posedge clk) begin
        if (imem_read)
            instr <= mem[addr];
    end
endmodule
```

Figure 6.2 :- Instruction Memory Module

6.3 Instruction Decoder

The Instruction Register is a 16-bit synchronous register that latches the instruction word output from the Instruction Memory on each clock edge when ir_write is asserted. It holds the instruction stable throughout the Decode, Read, Execute, Memory, and Writeback stages. Reset clears the register to 16'h0000.

Module :- instr_decoder.v

```
module instr_decoder (
    input [15:0] instr,
    output [3:0]  opcode,
    output [2:0]  rd, rs1, rs2,
    output [2:0]  reg_mem,
    output [8:0]  mem_addr
);
    assign opcode   = instr[15:12];
    assign rd       = instr[11:9];
    assign rs1      = instr[8:6];
    assign rs2      = instr[5:3];
    assign reg_mem  = instr[11:9];
    assign mem_addr = instr[8:0];
endmodule
```

Figure 6.3 :- Instruction Decoder Module

6.4 Arithmetic Logic Unit (ALU)

The ALU is a combinational unit supporting four operations selected by the 2-bit alu_op signal: **ADD** (00), **SUB** (01), **CMP** (10), and **MOV/passthrough** (11). For CMP, the result output is zeroed, and the three 1-bit flag outputs (g, l, z) are set based on the comparison of operands a and b. Only one flag is HIGH at a time.

alu_op	Operation	result output	Flags updated
2'b00	ADD	a + b (8-bit)	None
2'b01	SUB	a - b (8-bit)	None
2'b10	CMP	8'b0	Z, G, L set based on a vs b
2'b11	MOV	a (passthrough)	None

Module :- alu.v

```
module alu (
    input [7:0] a, b,
    input [1:0] alu_op,
    output reg [7:0] result,
    output reg g, l, z
);
    always @(*) begin
        g = 1'b0; l = 1'b0; z = 1'b0; result = 8'd0;

        case (alu_op)
            2'b00: result = a + b;
            2'b01: result = a - b;
            2'b10: begin
                result = 8'd0;
                if (a == b) begin
                    z = 1'b1; g = 1'b0; l = 1'b0;
                end else if (a > b) begin
                    g = 1'b1; z = 1'b0; l = 1'b0;
                end else begin
                    l = 1'b1; z = 1'b0; g = 1'b0;
                end
            end
            2'b11: result = a;
        endcase
    end
endmodule
```

Figure 6.4 :- ALU Module

6.5 Multiplier

A dedicated combinational 8x8 multiplier computes the full 16-bit product of two 8-bit operands. Only the lower 8 bits of the product are written to the register file during the MUL instruction. The multiplier is kept separate from the ALU to comply with the design constraint that the ALU must not perform multiplication.

For the test program: R5 = R3 x R2 = 10 x 4 = 40 = 0010_1000. The full 16-bit product is 0x0028; the lower byte 0x28 = 40 is written to R5.

Module :- multiplier.v

```
module multiplier (
    input [7:0] a, b,
    output [7:0] result
);
    wire [15:0] product;
    assign product = a * b;
    assign result  = product[7:0];
endmodule
```

Figure 6.5 :- Multiplier Module

6.6 Register File

The Register File contains 8 general-purpose 8-bit registers (R0–R7) implemented as a synchronous register array. It supports simultaneous combinational read of three source registers (rd_addr1, rd_addr2, rd_addr3) and synchronous write to one destination register when reg_write is asserted. The third read port (rd_data3) is used specifically for the STORE instruction, which requires the source register encoded in bits [11:9] of the M-type instruction format.

Module :- reg_file.v

```

module reg_file (
    input clk, rst, reg_write,
    input [2:0] rd_addr1, rd_addr2, rd_addr3,
    input [2:0] wr_addr,
    input [7:0] wr_data,
    output [7:0] rd_data1, rd_data2, rd_data3
);
    reg [7:0] regs [0:7];
    integer i;
    always @(posedge clk) begin
        if (rst) begin
            for (i = 0; i < 8; i = i + 1)
                regs[i] <= 8'd0;
        end else if (reg_write) begin
            regs[wr_addr] <= wr_data;
        end
    end
    assign rd_data1 = regs[rd_addr1];
    assign rd_data2 = regs[rd_addr2];
    assign rd_data3 = regs[rd_addr3];
endmodule
```

Figure 6.6 :- Register File Module

6.7 Instruction Register

The Instruction Register is a 16-bit synchronous register that latches the instruction word output from the Instruction Memory on each clock edge when ir_write is asserted. It holds the instruction stable throughout the Decode, Read, Execute, Memory, and Writeback stages. Reset clears the register to 16'h0000.

Module :- instr_reg.v

```
module instr_reg (
    input clk, ir_write, rst,
    input [15:0] ir_in,
    output reg [15:0] ir_out
);
    always @(posedge clk) begin
        if (rst)
            ir_out <= 16'h0000;
        else if (ir_write)
            ir_out <= ir_in;
    end
endmodule
```

Figure 6.7 :- Instruction Register Module

6.8 Data Memory

The Data Memory is a 512x8-bit synchronous RAM. Writes occur on the rising clock edge when mem_write is asserted. Reads are also registered — when mem_read is HIGH, the data at addr[8:0] is captured into read_data[7:0] on the next rising edge. The memory is pre-initialized with MEM[10]=6 and MEM[11]=4 using Verilog initial blocks.

Module :- data_mem.v

```
module data_mem (
    input clk, rst, mem_read, mem_write,
    input [8:0] addr,
    input [7:0] write_data,
    output reg [7:0] read_data
);
    reg [7:0] mem [0:511];
    integer i;

    initial begin
        for (i = 0; i < 512; i = i + 1)
            mem[i] = 8'd0;

        mem[10] = 8'd6;
        mem[11] = 8'd4;
    end

    always @(posedge clk) begin
        if (mem_write)
            mem[addr] <= write_data;
    end

    always @(posedge clk) begin
        if (rst)
            read_data <= 8'd0;
        else if (mem_read)
            read_data <= mem[addr];
    end
endmodule
```

Figure 6.8 :- Data Memory Module

6.9 DataPath

The Datapath module instantiates and interconnects all sub-modules: PC, Instruction Memory, Instruction Register, Instruction Decoder, Register File, ALU, Multiplier, and Data Memory. It accepts all control signals from the Control Unit and exposes the opcode field and flag outputs back to the Control Unit, forming the closed processor loop.

A set of shadow registers replicates the register file state for direct output, exposing reg0 through reg7 as 8-bit ports for display. This avoids adding complex multiplexed read-port logic to the register file itself.

Module :- datapath.v

```
module datapath (
    input clk, rst,
    input [1:0]  alu_op,
    input flag_write, halt, imem_read, ir_write, mem_read, mem_write, mul_en, pc_write, reg_write,
    input [1:0]  wb_sel,

    output flag_g, flag_l, flag_z,
    output [3:0]  opcode,
    output [7:0]  reg0, reg1, reg2, reg3, reg4, reg5, reg6, reg7
);

    wire [7:0] pc_out;
    wire [15:0] imem_instr, ir_out;
    pc u_pc (.clk(clk),.rst(rst),.pc_write(pc_write),.pc_out(pc_out));

    // Decoded fields
    wire [3:0]  dec_opcode;
    wire [2:0]  dec_rd, dec_rs1, dec_rs2;
    wire [2:0]  dec_reg_mem;
    wire [8:0]  dec_mem_addr;
    instr_decoder u_dec (.instr(ir_out),.opcode(dec_opcode),.rd(dec_rd),.rs1(dec_rs1),.rs2(dec_rs2),.reg_mem(dec_reg_mem),.mem_addr(dec_mem_addr));

    //Instruction Memory
    instr_mem u_imem (.clk(clk),.imem_read(imem_read),.addr(pc_out),.instr(imem_instr));

    // Register file outputs
    wire [7:0] rf_data1, rf_data2, rf_data3;
    reg [7:0] wb_data;
     reg_file u_rf (.clk(clk),.rst(rst),.reg_write(reg_write),.rd_addr1 (dec_rs1),.rd_addr2 (dec_rs2),.rd_addr3 (dec_reg_mem),
     .wr_addr  (dec_rd),.wr_data  (wb_data),.rd_data1 (rf_data1),.rd_data2 (rf_data2),.rd_data3 (rf_data3));

    // ALU outputs
    wire [7:0] alu_result;
    wire alu_g, alu_l, alu_z;
    alu u_alu (.a(rf_data1),.b(rf_data2),.alu_op(alu_op),.result(alu_result),.g(alu_g),.l(alu_l),.z(alu_z));

    // Multiplier output
```

Figure 6.9.1 :- Datapath Module

```
    // Multiplier output
    wire [7:0] mul_result;
    multiplier u_mul (.a(rf_data1),.b(rf_data2),.result(mul_result));

    //Instruction Register
    instr_reg u_ir (.clk(clk),.rst(rst),.ir_write(ir_write),.ir_in(imem_instr),.ir_out(ir_out));

    // Data Memory output
    wire [7:0] dmem_data;
    data_mem u_dmem (.clk(clk),.rst(rst),.mem_read(mem_read),.mem_write(mem_write),.addr(dec_mem_addr),.write_data(rf_data3),.read_data(dmem_data));

    // Flag registers
    reg flag_g_r, flag_l_r, flag_z_r;

    always @(*) begin
        case (wb_sel)
            2'b00:   wb_data = alu_result;
            2'b01:   wb_data = mul_result;
            2'b10:   wb_data = dmem_data;
            default: wb_data = alu_result;
        endcase
    end

    // Flag Registers
    always @(posedge clk) begin
        if (rst) begin
            flag_g_r <= 1'b0; flag_l_r <= 1'b0; flag_z_r <= 1'b0;
        end else if (flag_write) begin
            flag_g_r <= alu_g; flag_l_r <= alu_l; flag_z_r <= alu_z;
        end
    end

    assign flag_g = flag_g_r;
    assign flag_l = flag_l_r;
    assign flag_z = flag_z_r;

    // Opcode exposed to control unit (directly from IR)
```

Figure 6.9.2 :- Datapath Module (contd.)

```
    // Opcode exposed to control unit (directly from IR)
    assign opcode = ir_out[15:12];

    reg [7:0] shadow_regs [0:7];
    integer idx;

    always @(posedge clk) begin
        if (rst) begin
            for (idx = 0; idx < 8; idx = idx + 1)
                shadow_regs[idx] <= 8'd0;
        end else if (reg_write) begin
            shadow_regs[dec_rd] <= wb_data;
        end
    end

    assign reg0 = shadow_regs[0];
    assign reg1 = shadow_regs[1];
    assign reg2 = shadow_regs[2];
    assign reg3 = shadow_regs[3];
    assign reg4 = shadow_regs[4];
    assign reg5 = shadow_regs[5];
    assign reg6 = shadow_regs[6];
    assign reg7 = shadow_regs[7];

endmodule
```

Figure 6.9.3 :- Datapath Module (contd.)

6.10 Control Unit

The Control Unit is the central sequencer of the processor, implemented as a 7-state Moore Finite State Machine (FSM). It advances through the execution stages on each rising clock edge, generating all datapath control signals as combinational outputs of the current state and registered opcode.

State	Name	Active Control Signals
S_FETCH (0)	Fetch	imem_read=1, ir_write=1
S_DECODE (1)	Decode	opcode_reg = opcode (registered for timing)
S_READ (2)	Read Operands	Register file read (combinational, no control needed)
State	Name	Active Control Signals
S_EXECUTE (3)	Execute	alu_op set per opcode; mul_en=1 for MUL; flag_write=1 for CMP
S_MEMORY (4)	Memory Access	mem_read=1 for LOAD, wb_sel=WB_MEM; mem_write=1 for STORE
S_WRITEBACK (5)	Write Back	reg_write=1 (except CMP/STORE/NOP); pc_write=1 (except HALT)
S_HALT (6)	Halt	halt=1; pc_write=0; stays in S_HALT until rst

Write-Back Select (wb_sel):

wb_sel	Source	Used By
2'b00 (WB_ALU)	ALU result output	ADD, SUB, MOV, CMP (not written)
2'b01 (WB_MUL)	Multiplier result output	MUL
2'b10 (WB_MEM)	Data Memory read_data	LOAD

Module :- control_unit.v

```
module control_unit (
    input clk, rst,
    input [3:0] opcode,
    input flag_g, flag_l, flag_z,
    output reg pc_write, imem_read, ir_write, reg_write, mem_read, mem_write, mul_en, flag_write, halt,
    output reg [1:0] alu_op, wb_sel
);

    // Opcodes
    localparam OP_NOP = 4'b0000, OP_ADD = 4'b0001, OP_SUB = 4'b0010, OP_MUL = 4'b0011,
               OP_CMP = 4'b0100, OP_MOV = 4'b0101, OP_LOAD = 4'b0110, OP_STORE = 4'b0111, OP_HALT = 4'b1111;

    // ALU operations
    localparam ALU_ADD = 2'b00, ALU_SUB = 2'b01, ALU_CMP = 2'b10, ALU_MOV = 2'b11;

    // Write-back select
    localparam WB_ALU = 2'b00, WB_MUL = 2'b01, WB_MEM = 2'b10;

    // FSM States
    localparam S_FETCH = 3'd0, S_DECODE = 3'd1, S_READ = 3'd2, S_EXECUTE = 3'd3, S_MEMORY = 3'd4, S_WRITEBACK = 3'd5, S_HALT = 3'd6;

    reg [2:0] state, next_state;
    reg [3:0] opcode_reg;
    reg [1:0] alu_op_reg;

    always @(posedge clk) begin
        if (rst) begin
            state <= S_FETCH;
            opcode_reg <= OP_NOP;
            alu_op_reg <= ALU_ADD;
        end else begin
            state <= next_state;
            if (state == S_DECODE)
                opcode_reg <= opcode;

            if(state == S_EXECUTE) begin
                case (opcode_reg)
```

Figure 6.10.1 :- Control Unit Module

```
                case (opcode_reg)
                    OP_ADD:  alu_op_reg <= ALU_ADD;
                    OP_SUB:  alu_op_reg <= ALU_SUB;
                    OP_CMP:  alu_op_reg <= ALU_CMP;
                    OP_MOV:  alu_op_reg <= ALU_MOV;
                    default: alu_op_reg <= ALU_ADD;
                endcase
            end
        end
    end

    always @(*) begin
        case (state)
            S_FETCH:     next_state = S_DECODE;
            S_DECODE:    next_state = S_READ;
            S_READ:      next_state = S_EXECUTE;
            S_EXECUTE:   next_state = S_MEMORY;
            S_MEMORY:    next_state = S_WRITEBACK;
            S_WRITEBACK: next_state = (opcode_reg == OP_HALT) ? S_HALT : S_FETCH;
            S_HALT:      next_state = S_HALT;
            default:     next_state = S_FETCH;
        endcase
    end

    always @(*) begin
        pc_write   = 1'b0;
        imem_read  = 1'b0;
        ir_write   = 1'b0;
        reg_write  = 1'b0;
        mem_read   = 1'b0;
        mem_write  = 1'b0;
        mul_en     = 1'b0;
        flag_write = 1'b0;
        halt       = 1'b0;
        alu_op     = ALU_ADD;
        wb_sel     = WB_ALU;

```

Figure 6.10.2 :- Control Unit Module (Contd.)

```
        case (state)

            S_FETCH: begin
                imem_read = 1'b1;
                ir_write  = 1'b1;
            end

            S_EXECUTE: begin
                case(opcode_reg)
                    OP_ADD: alu_op = ALU_ADD;
                    OP_SUB: alu_op = ALU_SUB;
                    OP_MUL: begin
                        mul_en = 1'b1;
                        wb_sel = WB_MUL;
                    end
                    OP_CMP: begin
                        alu_op     = ALU_CMP;
                        flag_write = 1'b1;
                    end
                    OP_MOV: alu_op = ALU_MOV;
                endcase
            end

            S_MEMORY: begin
                case (opcode_reg)
                    OP_LOAD: begin
                        mem_read = 1'b1;
                        wb_sel   = WB_MEM;
                    end
                    OP_STORE: mem_write = 1'b1;
                endcase
            end

            S_WRITEBACK: begin
                case (opcode_reg)
                    OP_ADD, OP_SUB: begin
                        reg_write = 1'b1;
                        alu_op    = alu_op_reg;
                        wb_sel    = WB_ALU;
                    end
                    OP_MUL: begin
                        reg_write = 1'b1;
                        wb_sel    = WB_MUL;
                    end
                    OP_MOV: begin
                        reg_write = 1'b1;
                        alu_op    = alu_op_reg;
                        wb_sel    = WB_ALU;
                    end
                    OP_LOAD: begin
                        reg_write = 1'b1;
                        wb_sel    = WB_MEM;
                    end
                    OP_HALT: halt = 1'b1;
                endcase
                if (opcode_reg != OP_HALT)
                    pc_write = 1'b1;
            end

            S_HALT: halt = 1'b1;
        endcase
    end
endmodule
```

Figure 6.10.3 :- Control Unit Module (Contd.)

6.11 Register Selection

The reg_select module is used to display the contents of one of the eight registers (R0–R7) on the FPGA LEDs. It selects the desired register based on a 3-bit input switch (SW). This allows easy observation and debugging of register values during processor execution.

Module :- reg_select.v

```
module reg_select(
    input [7:0] reg0,reg1,reg2,reg3,reg4,reg5,reg6,reg7,
    input [2:0] SW,
    output [7:0] LED
    );

    reg [7:0] LED_reg;
    always @(*) begin
        case (SW)
            3'd0: LED_reg = reg0;
            3'd1: LED_reg = reg1;
            3'd2: LED_reg = reg2;
            3'd3: LED_reg = reg3;
            3'd4: LED_reg = reg4;
            3'd5: LED_reg = reg5;
            3'd6: LED_reg = reg6;
            3'd7: LED_reg = reg7;
        endcase
    end
    assign LED = LED_reg;
endmodule
```

Figure 6.11 :- Register Selection Module

6.12 Top Module – RISC Processor

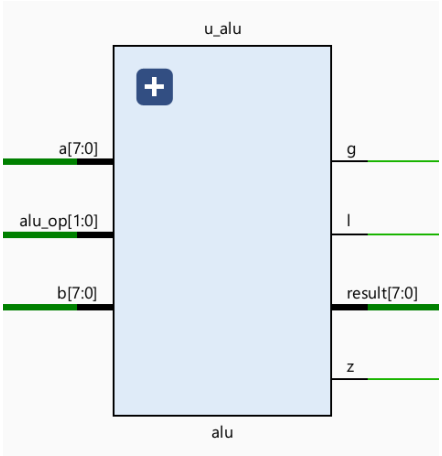
The top-level module risc_processor instantiates the Control Unit (u_cu), Datapath (u_dp), and Register Select (u_reg_sel) modules. It connects the bidirectional opcode and flag signals between the Control Unit and Datapath, and maps the switch-selected register output to the LED array for FPGA demonstration. The Register Select module (reg_select) is a 3-to-8 multiplexer driven by the on-board slide switches SW[2:0], displaying the value of register R0–R7 on the 8 LEDs. Flags flag_Z, flag_G, flag_L are mapped to dedicated LEDs on the board.

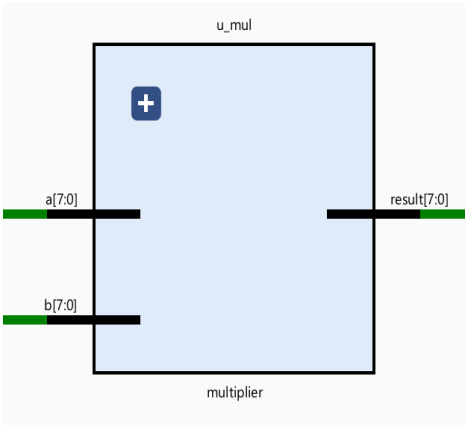
Module :- risc_processor.v

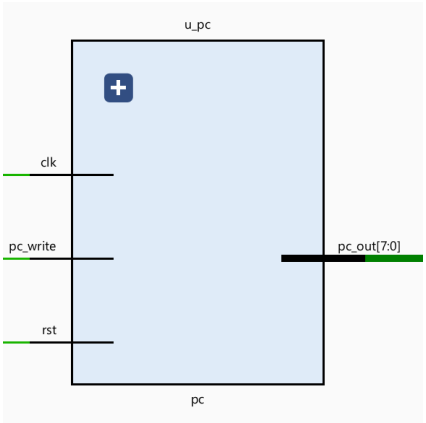
```
module risc_processor (
    input clk, rst,
    input [2:0] SW,
    output [7:0] LED,
    output flag_Z, flag_G, flag_L

);
    wire [1:0] alu_op;
    wire flag_write, halt, imem_read, ir_write, mem_read, mem_write, mul_en, pc_write, reg_write;
    wire [1:0] wb_sel;

    wire [3:0] opcode;
    wire flag_g_int, flag_l_int, flag_z_int;
    wire [7:0] reg0_out,reg1_out,reg2_out,reg3_out,reg4_out,reg5_out,reg6_out,reg7_out;

    control_unit u_cu (.clk(clk),.rst(rst),.opcode(opcode),.flag_g(flag_g_int),.flag_l(flag_l_int),.flag_z(flag_z_int),.pc_write(pc_write),.imem_read(imem_read),
        .ir_write(ir_write),.reg_write(reg_write),.mem_read(mem_read),.mem_write(mem_write),.mul_en(mul_en),.flag_write(flag_write),.halt(halt),.alu_op(alu_op),.wb_sel(wb_sel));

    datapath u_dp (.clk(clk),.rst(rst),.alu_op(alu_op),.flag_write(flag_write),.halt(halt),.imem_read(imem_read),.ir_write(ir_write),.mem_read(mem_read),
        .mem_write(mem_write),.mul_en(mul_en),.pc_write(pc_write),.reg_write(reg_write),.wb_sel(wb_sel),.flag_g(flag_g_int),.flag_l(flag_l_int),.flag_z(flag_z_int),
        .opcode(opcode),.reg0(reg0_out),.reg1(reg1_out),.reg2(reg2_out),.reg3(reg3_out),.reg4(reg4_out),.reg5(reg5_out),.reg6(reg6_out),.reg7(reg7_out));

    assign flag_Z = flag_z_int;
    assign flag_G = flag_g_int;
    assign flag_L = flag_l_int;

    reg_select u_reg_sel(.reg0(reg0_out),.reg1(reg1_out),.reg2(reg2_out),.reg3(reg3_out),.reg4(reg4_out),.reg5(reg5_out),.reg6(reg6_out),.reg7(reg7_out),.SW(SW),.LED(LED));
endmodule
```

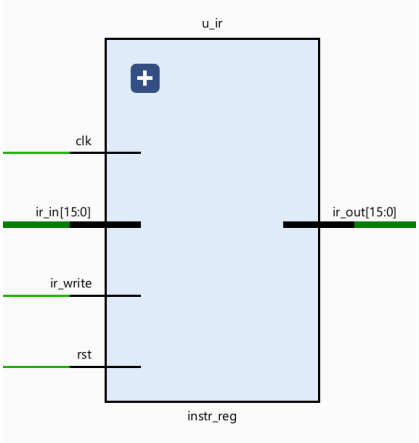
Figure 6.12 :- Top Module

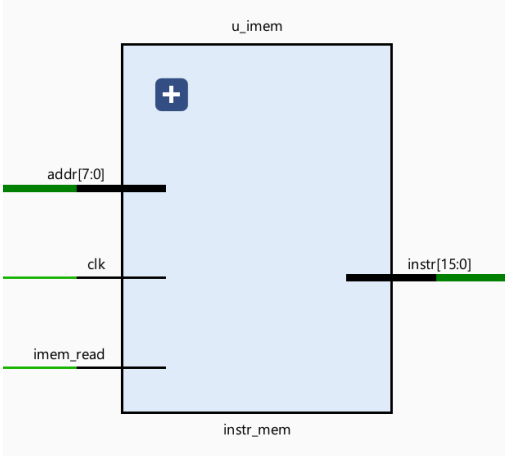
7. Modules Synthesized Block Diagram

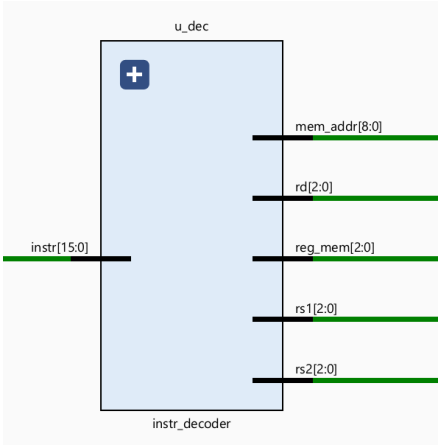


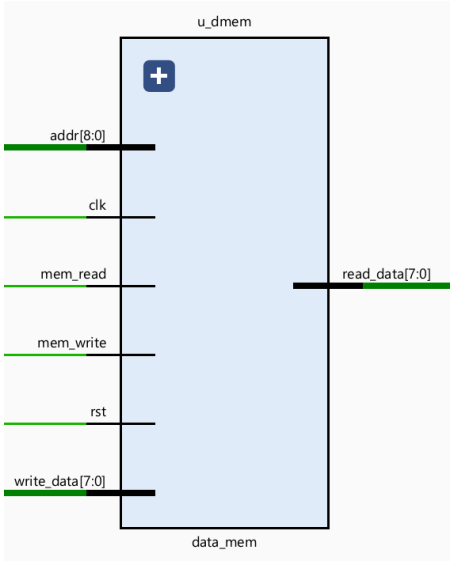


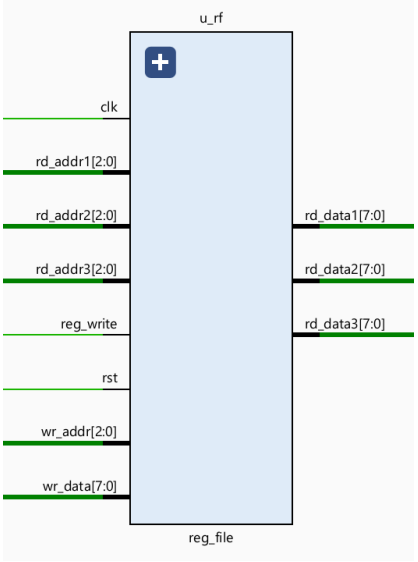


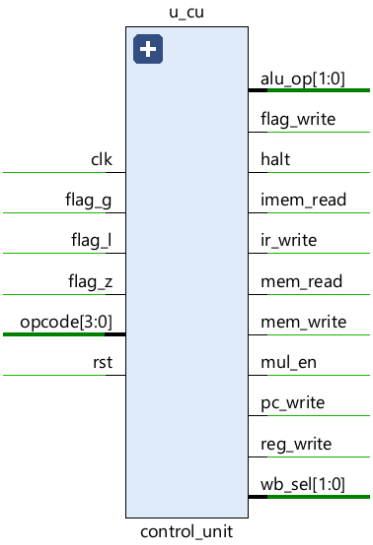


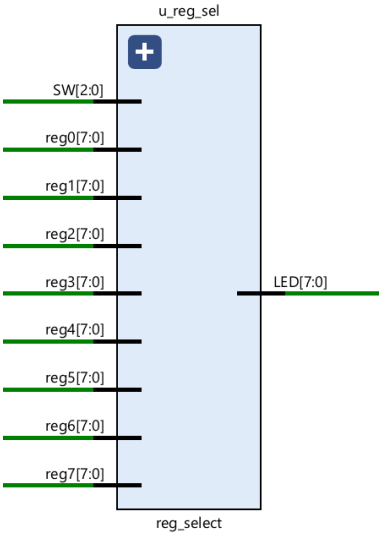












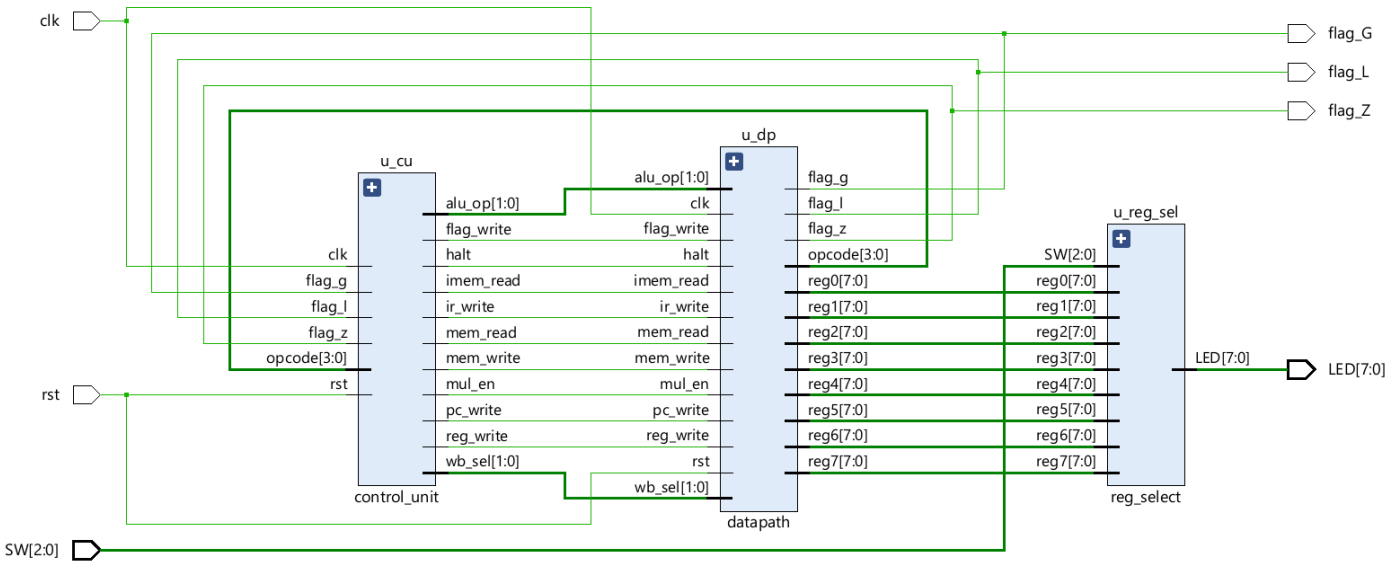
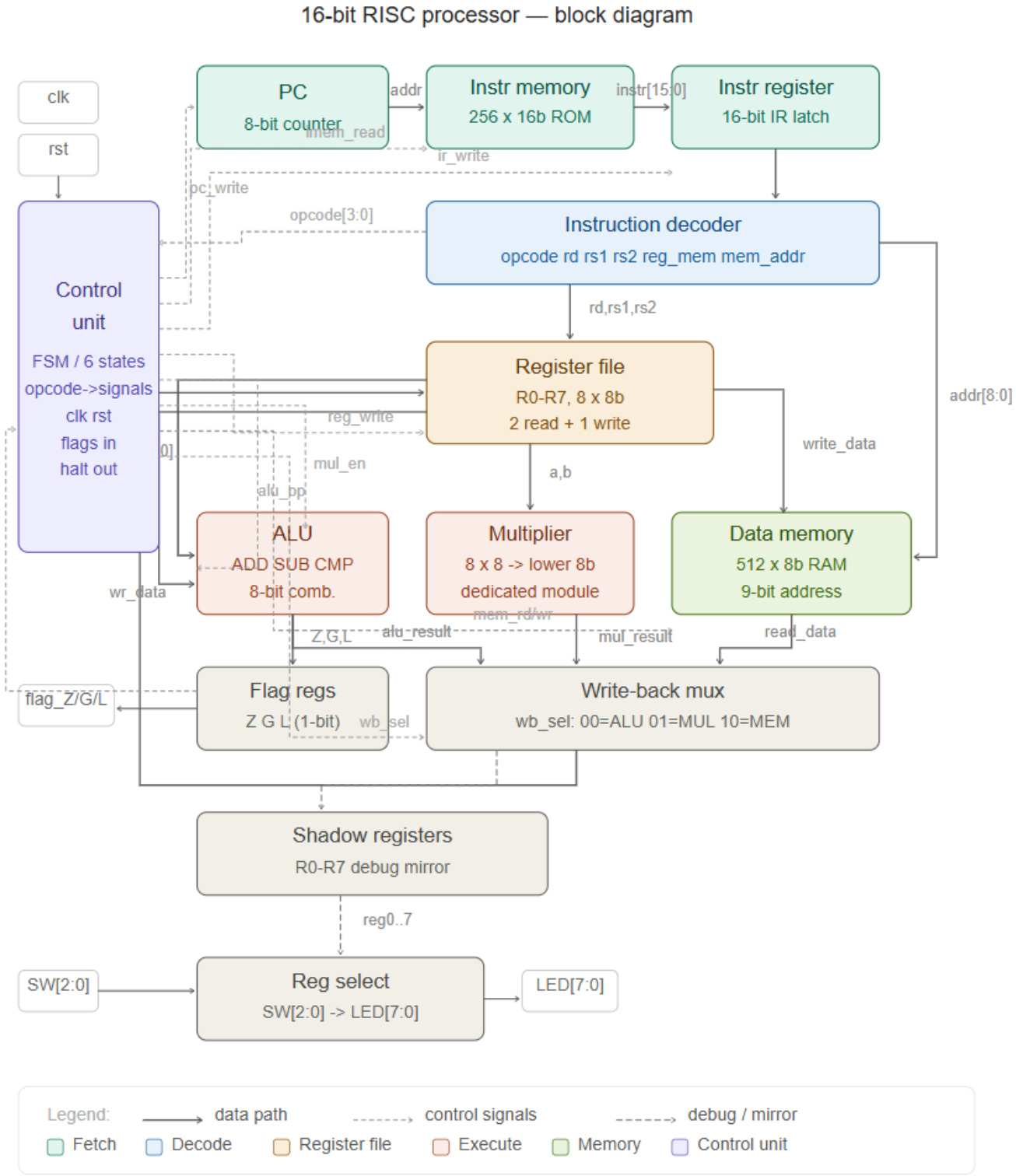
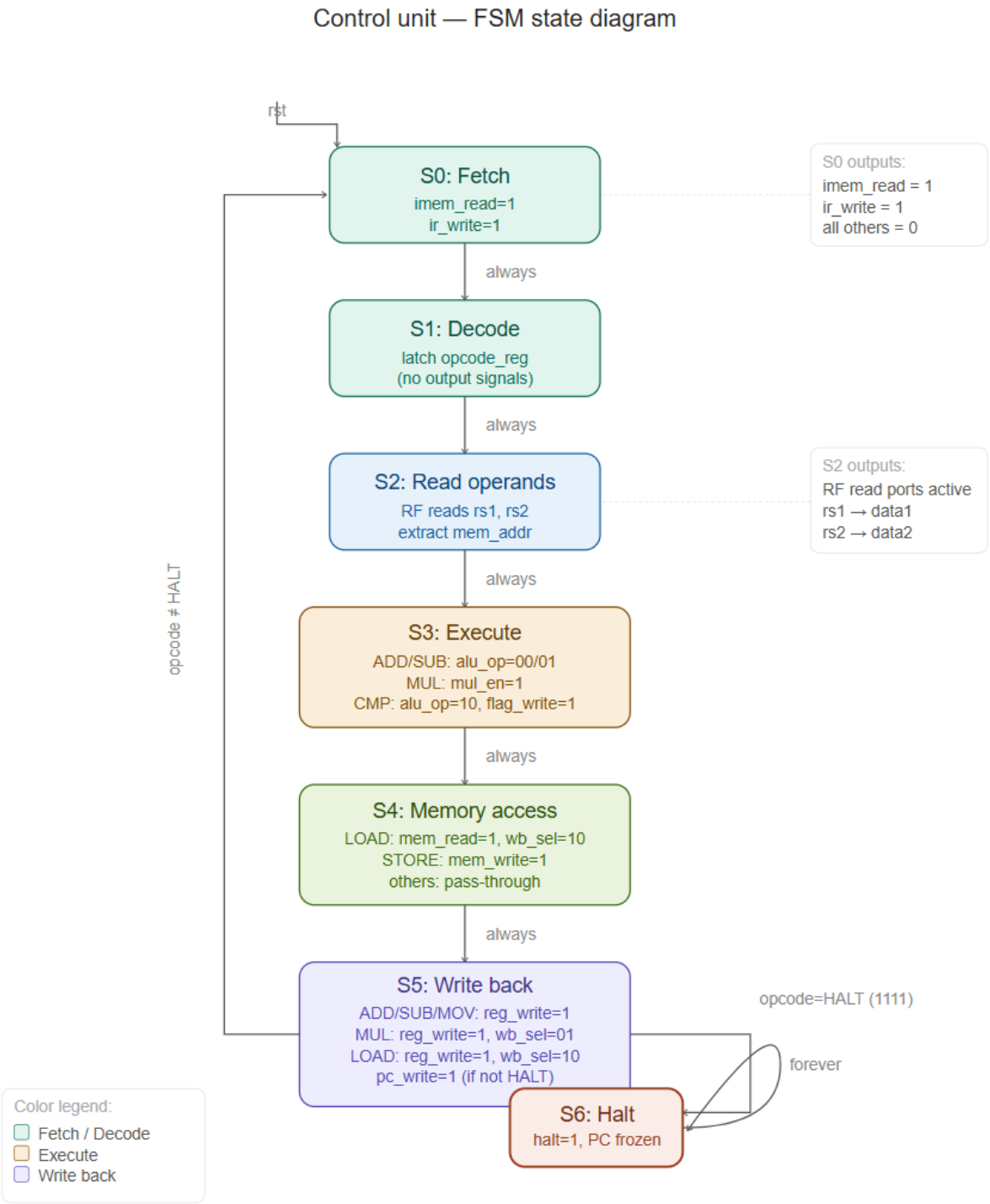


Figure 7 :- Top Module

8. Block Diagram



9. State Diagram



10. Mandatory Test Program

10.1 Initial Data Memory State

Address	Value (Decimal)	Value (8-bit Binary)	Hex
MEM[10]	6	0000_0110	0x06
MEM[11]	4	0000_0100	0x04

10.2 Instruction Sequence

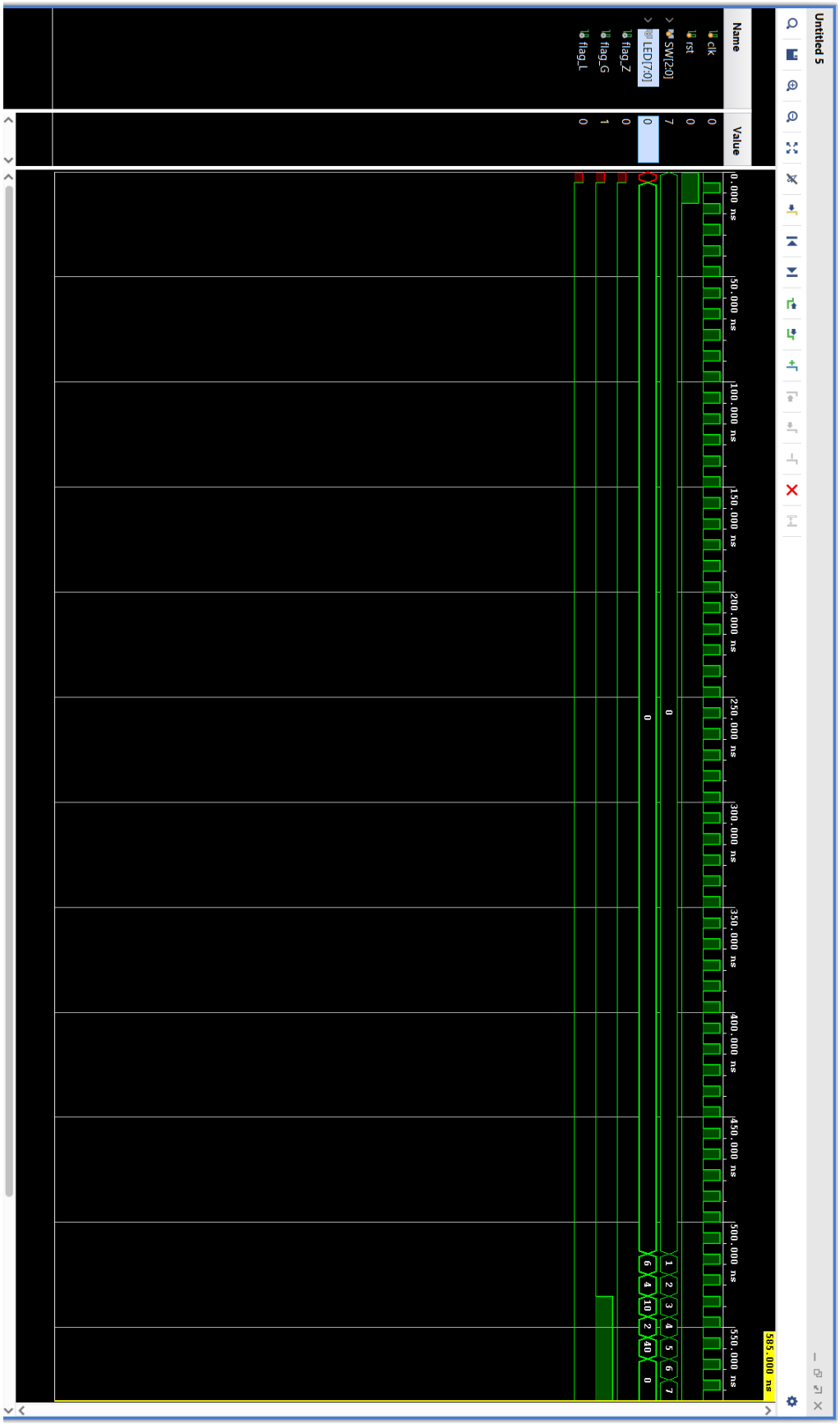
PC	Instruction	16-bit Encoding	Operation
0	LOAD R1, 10	0110_001_000001010 = 0x620A	R1 = MEM[10] = 6
1	LOAD R2, 11	0110_010_000001011 = 0x640B	R2 = MEM[11] = 4
2	ADD R3, R1, R2	0001_011_001_010_000 = 0x1650	R3 = R1 + R2 = 10
3	SUB R4, R1, R2	0010_100_001_010_000 = 0x2850	R4 = R1 - R2 = 2
4	MUL R5, R3, R2	0011_101_011_010_000 = 0x3AD0	R5 = R3 x R2 = 40
5	STORE R5, 20	0111_101_000010100 = 0x7A14	MEM[20] = R5 = 40
6	CMP R3, R4	0100_000_011_100_000 = 0x40E0	R3>R4 => G=1, Z=0, L=0
7	HALT	1111_000_000_000_000 = 0xF000	Freeze PC; stop execution

10.3 Expected Results after HALT

Register / Location	Expected Value	Binary	Source Instruction	Verification
R1	6	0000_0110	LOAD R1, 10	LED = 0x06
R2	4	0000_0100	LOAD R2, 11	LED = 0x04
R3	10	0000_1010	ADD R3, R1, R2	LED = 0x0A
R4	2	0000_0010	SUB R4, R1, R2	LED = 0x02
R5	40	0010_1000	MUL R5, R3, R2	LED = 0x28
MEM[20]	40	0010_1000	STORE R5, 20	LOAD back verify
flag_G	1 (HIGH)	—	CMP R3, R4	Dedicated LED = ON
flag_Z	0 (LOW)	—	CMP R3, R4	Dedicated LED = OFF
flag_L	0 (LOW)	—	CMP R3, R4	Dedicated LED = OFF

11. Simulation Results

The processor was simulated using Vivado Simulator (xsim) with the provided testbench. The testbench applies a synchronous reset for 15 ns, then releases reset and runs for 500 ns to allow all 8 instructions to execute. After HALT, the register select switch is stepped through R1 to R7 to verify each register output.



12. FPGA Synthesis & Implementation

The complete processor design was synthesized and implemented using Xilinx Vivado Design Suite targeting the **Boolean Board (XC7S50-CSGA324-1)**. The XDC constraints file maps the top-level ports to the appropriate FPGA pins.

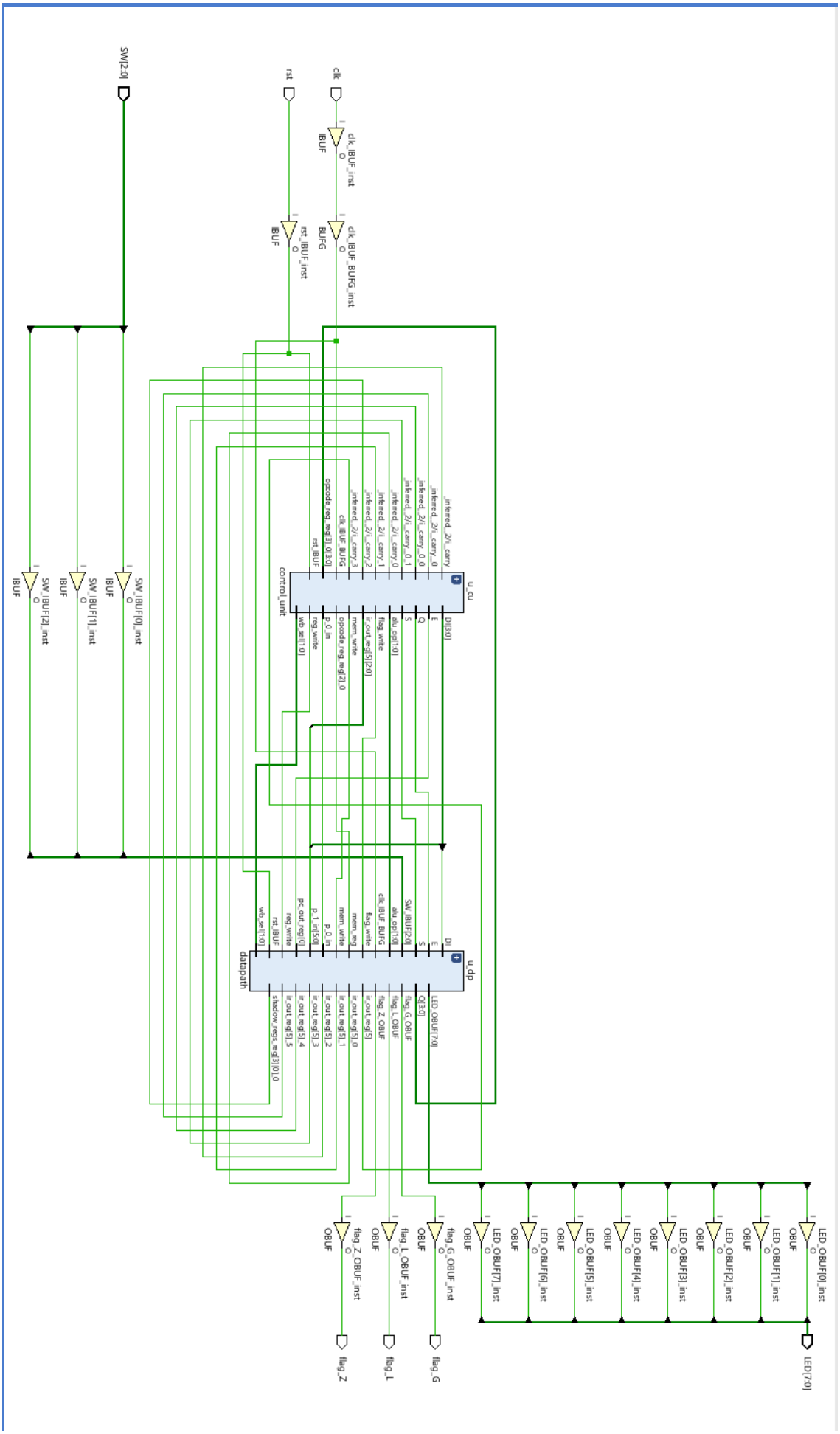
12.1 Pin Assignment (XDC Mapping)

Signal	FPGA Pin	I/O Standard	Description
clk	F14	LVCMOS33	100 MHz system clock from on-board oscillator
rst	J2	LVCMOS33	Active-HIGH reset mapped to on-board push button
SW[0]	V2	LVCMOS33	Slide switch 0 — register select bit 0
SW[1]	U2	LVCMOS33	Slide switch 1 — register select bit 1
SW[2]	U1	LVCMOS33	Slide switch 2 — register select bit 2
LED[0]	G1	LVCMOS33	Register value bit 0
LED[1]	G2	LVCMOS33	Register value bit 1
LED[2]	F1	LVCMOS33	Register value bit 2
LED[3]	F2	LVCMOS33	Register value bit 3
LED[4]	E1	LVCMOS33	Register value bit 4
LED[5]	E2	LVCMOS33	Register value bit 5
LED[6]	E3	LVCMOS33	Register value bit 6
LED[7]	E5	LVCMOS33	Register value bit 7
flag_L	A3	LVCMOS33	Less flag — dedicated LED
flag_G	B4	LVCMOS33	Greater flag — dedicated LED
flag_Z	A4	LVCMOS33	Zero/Equal flag — dedicated LED

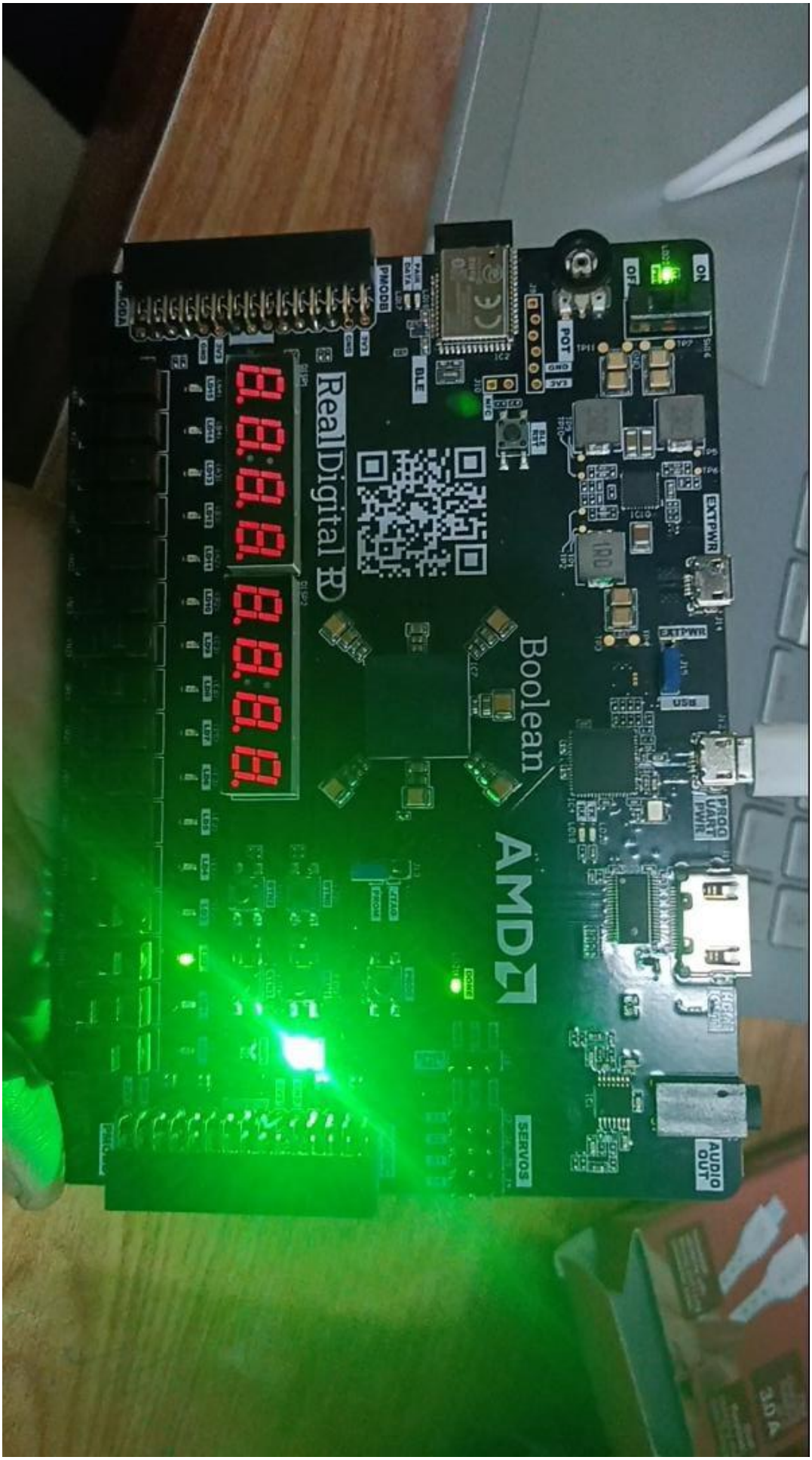
12.2 Constraints File (risc.xdc)

```
# clk input is from the 100 MHz oscillator on Boolean board
create_clock -period 10.000 -name gclk [get_ports clk]
set_property -dict {PACKAGE_PIN F14 IOSTANDARD LVCMOS33} [get_ports {clk}]

# Set Bank 0 voltage
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

# On-board Slide Switches
set_property -dict {PACKAGE_PIN V2 IOSTANDARD LVCMOS33} [get_ports {SW[0]}]
set_property -dict {PACKAGE_PIN U2 IOSTANDARD LVCMOS33} [get_ports {SW[1]}]
set_property -dict {PACKAGE_PIN U1 IOSTANDARD LVCMOS33} [get_ports {SW[2]}]

# On-board LEDs
set_property -dict {PACKAGE_PIN G1 IOSTANDARD LVCMOS33} [get_ports {LED[0]}]
set_property -dict {PACKAGE_PIN G2 IOSTANDARD LVCMOS33} [get_ports {LED[1]}]
set_property -dict {PACKAGE_PIN F1 IOSTANDARD LVCMOS33} [get_ports {LED[2]}]
set_property -dict {PACKAGE_PIN F2 IOSTANDARD LVCMOS33} [get_ports {LED[3]}]
set_property -dict {PACKAGE_PIN E1 IOSTANDARD LVCMOS33} [get_ports {LED[4]}]
set_property -dict {PACKAGE_PIN E2 IOSTANDARD LVCMOS33} [get_ports {LED[5]}]
set_property -dict {PACKAGE_PIN E3 IOSTANDARD LVCMOS33} [get_ports {LED[6]}]
set_property -dict {PACKAGE_PIN E5 IOSTANDARD LVCMOS33} [get_ports {LED[7]}]
#set_property -dict {PACKAGE_PIN E6 IOSTANDARD LVCMOS33} [get_ports {LED[8]}]
#set_property -dict {PACKAGE_PIN C3 IOSTANDARD LVCMOS33} [get_ports {LED[9]}]
#set_property -dict {PACKAGE_PIN B2 IOSTANDARD LVCMOS33} [get_ports {LED[10]}]
#set_property -dict {PACKAGE_PIN A2 IOSTANDARD LVCMOS33} [get_ports {LED[11]}]
#set_property -dict {PACKAGE_PIN B3 IOSTANDARD LVCMOS33} [get_ports {LED[12]}]
set_property -dict {PACKAGE_PIN A3 IOSTANDARD LVCMOS33} [get_ports flag_L]
set_property -dict {PACKAGE_PIN B4 IOSTANDARD LVCMOS33} [get_ports flag_G]
set_property -dict {PACKAGE_PIN A4 IOSTANDARD LVCMOS33} [get_ports flag_Z]

# On-board Buttons
set_property -dict {PACKAGE_PIN J2 IOSTANDARD LVCMOS33} [get_ports rst]
```

12.3 Synthesis Result



12.4 FPGA Hardware Demonstration



13. Conclusion

This laboratory assignment successfully demonstrated the complete design, simulation, and FPGA deployment of a custom 16-bit RISC processor on the Boolean Board (Xilinx Spartan-7 XC7S50). The processor implements a 9-instruction custom ISA over an 8-bit datapath with a 16-bit fixed instruction width.

The FSM-based Control Unit correctly sequences the processor through six execution stages (Fetch, Decode, Read, Execute, Memory, Writeback) for every instruction, transitioning to a HALT state upon executing the HALT opcode. All nine mandatory modules — PC, Instruction Memory, Instruction Register, Register File, ALU, Multiplier, Data Memory, Instruction Decoder, and Control Unit — were independently designed, simulated, and integrated.

The mandatory test program executed correctly in both simulation (Vivado xsim) and on hardware (Boolean Board). All expected post-HALT register values (R1=6, R2=4, R3=10, R4=2, R5=40) and memory contents (MEM[20]=40) were verified. The CMP instruction correctly set flag_G=1, flag_Z=0, flag_L=0, confirmed by the dedicated flag LEDs on the board.

This project reinforced core concepts of computer architecture including ISA design, datapath construction, synchronous control logic, register file organization, memory-mapped I/O, and FPGA synthesis flow.